

Deep Reinforcement Learning on Constrained Hardware: Solving Three Environments with 2 CPU Cores and No GPU

Turing
RTSS Board
turingrtss@gmail.com

September 2026

Abstract

We train a Deep Q-Network (DQN) agent on three progressively complex OpenAI Gymnasium environments using a single ARM64 server with 2 CPU cores, 12 GB RAM, and no GPU. All three environments—CartPole-v1, Acrobot-v1, and LunarLander-v3—are solved within standard convergence thresholds. Total training time across all three is 59.7 minutes with a peak memory footprint of 341 MB. These results establish a practical lower bound on the hardware required for classical RL benchmarks and demonstrate that meaningful reinforcement learning experiments are accessible without GPU infrastructure.

1 Introduction

Reinforcement learning research is commonly associated with large-scale compute: GPU clusters for Atari [1], TPU pods for Go [2], and multi-day training runs for robotics [3]. This creates an implicit assumption that RL experiments require expensive hardware, which may discourage researchers and practitioners with limited resources.

However, many foundational RL benchmarks—the environments used to validate new algorithms and implementations—are computationally modest. The question of *how modest* has received little direct attention. Published papers rarely report wall-clock training times or memory usage on consumer hardware, focusing instead on sample efficiency and asymptotic performance.

This paper asks a concrete question: what can a DQN agent learn on a 2-core ARM CPU with no GPU? We train on three standard environments of increasing complexity and report wall-clock time, memory consumption, convergence speed, and training throughput, providing a practical reference for RL practitioners working under hardware constraints.

2 Methods

2.1 Hardware

All experiments were conducted on a single Oracle Cloud free-tier instance:

- **CPU:** 2-core ARM Neoverse-N1 (aarch64)
- **RAM:** 12 GB
- **GPU:** None
- **OS:** Ubuntu 22.04
- **Framework:** PyTorch 2.14.0 (CPU build)

2.2 Environments

Three Gymnasium [4] environments were selected to represent increasing complexity:

Table 1: Environment specifications.

Environment	Obs. Dim	Act. Dim	Solve Threshold	Difficulty
CartPole-v1	4	2	≥ 475	Simple
Acrobot-v1	6	3	≥ -100	Medium
LunarLander-v3	8	4	≥ 200	Hard

Convergence is defined as achieving the solve threshold averaged over a rolling window of 100 episodes.

2.3 Agent Architecture

We implement a standard DQN [5] with the following components:

- **Network:** Two hidden layers (ReLU activation) with configurable width. CartPole uses 64 hidden units; Acrobot and LunarLander use 128.
- **Target network:** Updated every 10 episodes via hard copy.
- **Replay buffer:** Capacity 50,000 transitions, uniform sampling.
- **Exploration:** ϵ -greedy with exponential decay ($\epsilon_{\text{start}} = 1.0$, $\epsilon_{\text{end}} = 0.01$, decay factor 0.995 per episode).
- **Optimizer:** Adam. Learning rate 10^{-3} for CartPole, 5×10^{-4} for Acrobot and LunarLander.
- **Batch size:** 64.
- **Discount factor:** $\gamma = 0.99$.

No advanced techniques (prioritized replay, double DQN, dueling networks, n-step returns) were employed. The intent is to measure baseline DQN performance under hardware constraints, not to optimize sample efficiency.

2.4 Measurement

For each environment we record:

- **Wall-clock time** (seconds) from first to last episode.
- **Peak RSS memory** (MB) via `psutil`.
- **Convergence episode:** first episode at which the rolling 100-episode average meets the solve threshold.
- **Training throughput:** episodes per second.

3 Results

3.1 Convergence

All three environments were solved within their allocated episode budgets.

Table 2: Training results across all three environments.

Env.	Conv. Ep.	Final Avg.	Time (s)	Time (min)	Mem (MB)	Ep/s
CartPole-v1	975	461.1	607	10.1	324	1.6
Acrobot-v1	728	-80.2	768	12.8	324	2.0
LunarLander-v3	677	225.1	2,208	36.8	341	0.9
Total	—	—	3,583	59.7	341	—

3.2 Training Dynamics

CartPole-v1 exhibited gradual improvement from a random baseline (avg. reward 40 at episode 100) to near-optimal performance (avg. reward 461 at episode 1,000). Convergence required 975 episodes. The loss increased throughout training, reaching 13.2 by the final episode, consistent with the growing magnitude of Q-value estimates in long-horizon episodes.

Acrobot-v1 converged fastest in terms of episodes (728), improving from -485 (near-random) to -80 (well above the -100 threshold). The environment’s shorter episodes (max 500 steps) resulted in higher throughput (2.0 ep/s) despite the larger network.

LunarLander-v3 was the most computationally demanding due to its higher-dimensional state and action spaces and longer episodes. It converged at episode 677 but required 36.8 minutes of wall-clock time. Post-convergence performance continued improving, reaching an average reward of 270 by episode 1,700 before slight degradation in the final 300 episodes.

3.3 Memory Profile

Peak memory usage was remarkably stable across all three environments. CartPole and Acrobot both peaked at 324 MB, with LunarLander adding only 17 MB more (341 MB). The replay buffer (50,000 transitions $\times$ approximately 100 bytes each $\approx$ 5 MB) is a small fraction of total memory. The dominant cost is the PyTorch runtime itself.

4 Discussion

4.1 Practical Implications

The central result is that all three classical RL benchmarks are solvable in under an hour on hardware that costs nothing (Oracle Cloud free tier) or very little (any ARM single-board computer with 1 GB+ RAM). The 341 MB peak memory footprint means a Raspberry Pi 4 (1 GB model) could run these experiments.

This has practical implications for:

- **Education:** Students can run RL experiments without cloud GPU credits.
- **Prototyping:** Algorithm variants can be tested locally before scaling.
- **Reproducibility:** CPU-only experiments are more reproducible than GPU experiments, which are sensitive to driver versions, CUDA compatibility, and nondeterministic GPU operations.

4.2 Where the Hardware Wall Is

These three environments represent the “easy” end of the RL benchmark spectrum. The hardware wall for this setup likely appears at:

- **Image-based observations** (e.g., Atari): Convolutional networks on 84×84 frames with no GPU would be prohibitively slow.
- **Large replay buffers**: 1M+ transitions for Atari-scale experiments would consume 2–4 GB of RAM.
- **Parallel environment execution**: Algorithms like A3C [6] or PPO [7] benefit from running many environment instances simultaneously. Two CPU cores limit parallelism severely.
- **Long training horizons**: MuJoCo continuous control tasks [8] typically require millions of timesteps, which at 0.9 ep/s would take days.

A follow-up study will test these boundaries empirically by attempting Atari environments on this hardware.

4.3 Limitations

1. Only one algorithm (DQN) was tested. Policy gradient methods (PPO, A2C) may have different hardware profiles.
2. No hyperparameter optimization was performed. Better hyperparameters could reduce convergence time.
3. The environments tested have low-dimensional state spaces. The results do not generalize to image-based or continuous-action environments.
4. Single-seed results. Variance across seeds is not reported.

5 Conclusion

A standard DQN agent solves CartPole-v1, Acrobot-v1, and LunarLander-v3 in a combined 59.7 minutes using 341 MB of RAM on a 2-core ARM CPU with no GPU. These results demonstrate that classical RL benchmarks do not require GPU infrastructure and can be run on free-tier cloud instances or consumer single-board computers. The hardware wall for this setup lies at image-based observations and million-step training horizons, which we plan to characterize in future work.

Data and Code Availability

All code, training scripts, and raw results are available at <https://github.com/turingrtss/vulndetect>. The experiment requires only PyTorch (CPU), Gymnasium, and psutil.

References

- [1] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.
- [2] D. Silver *et al.*, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016.

- [3] J. Hwangbo *et al.*, “Learning agile and dynamic motor skills for legged robots,” *Science Robotics*, vol. 4, no. 26, Jan. 2019.
- [4] M. Towers *et al.*, “Gymnasium: A standard interface for reinforcement learning environments,” arXiv:2407.17032, 2024.
- [5] V. Mnih *et al.*, “Playing Atari with deep reinforcement learning,” arXiv:1312.5602, 2013.
- [6] V. Mnih *et al.*, “Asynchronous methods for deep reinforcement learning,” in *Proc. 33rd Int. Conf. Mach. Learning (ICML)*, 2016, pp. 1928–1937.
- [7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
- [8] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2012, pp. 5026–5033.